SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT2 - VM Configuration and Security Evidence Workbook

Guided practical workbook for Azure VM, local VM, Docker container, security checks, recovery evidence and GitHub submission

Assessment Metadata

Course Code	CSA1517
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT2 - VM Configuration and Security Evidence
Submission Type	Individual digital workbook based on the same capstone title used in CO1 and CO2 AT1
Faculty	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Submission Mode	Completed workbook, evidence screenshots and GitHub repository link through Google Classroom

How This Workbook Works

- Use the same capstone title carried from CO1 and CO2 AT1.
- Read each procedure block, perform the action, verify the checkpoint and then fill the response table.
- Use Azure first when account activation is available. Use the local VM path when Azure is blocked. Use Docker for container comparison. Use simulation only when practical deployment is blocked.
- Screenshots must show the student's actual configuration or verified simulation work. Hide credentials, keys, tokens and sensitive account details.
- Upload the completed workbook and evidence to GitHub, then submit the GitHub repository link through Google Classroom.

Virtualization Tracks for CO2 AT2

Azure is the preferred cloud path. Local VM and container tracks are included for hands-on resilience.

Track A - Azure Linux VM

Preferred cloud evidence: subscription, VM, SSH, NGINX/test service, firewall, shutdown

Track B - Local VM

VirtualBox or VMware with Tiny Core, Alpine, Debian netinst or Ubuntu Server

Track C - Docker / Podman

Run a lightweight service, map a port, inspect logs, compare with VM

Track D - Simulation

Used only if deployment is blocked; must include technical configuration and security evidence

Required common output: completed workbook, screenshots, command outputs, security choices, README.md, GitHub repository link and GCR submission.

Technical Orientation

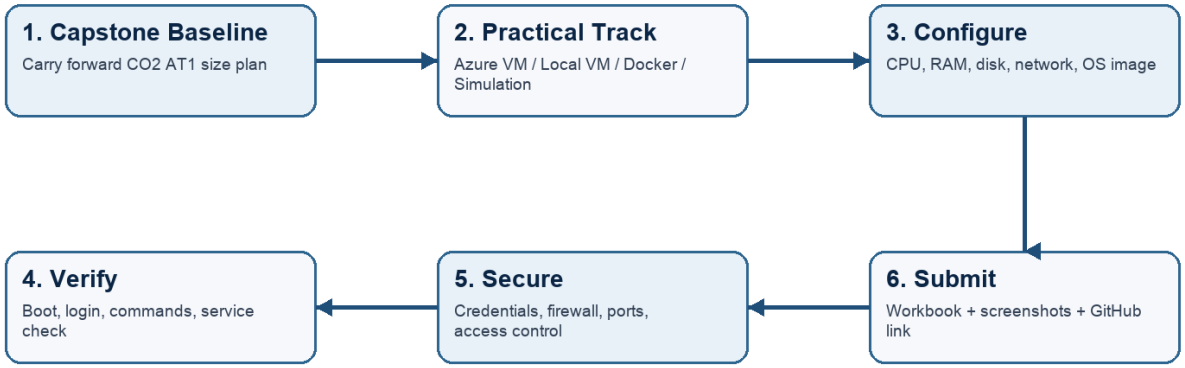
The practical work is not limited to clicking through screens. Each checkpoint requires observation: what appeared on screen, why it appeared, what it proves and how it connects to the capstone infrastructure plan.

Part 1: Student and Capstone Baseline

Student Name	DENVAR JOE H
Register Number	192511438
Class / Slot	Slot C
Capstone Title	HostelMate Green Cloud: Sustainable Hostel Management with Energy-Efficient SLA Optimization and Geo-Tagged Complaint Intelligence
CO2 AT1 Recommended VM Size	vCPU: 2 RAM: 4 GB Disk: 20-30 GB (confirm against your actual CO2 AT1 answer)
Selected Practical Track	Simulation (Track D) - update to Azure VM/Local VM/Docker once practical work is completed
GitHub Repository Link	https://github.com/Denvarj/CLOUD-COMPUTING-AND-BIG-DATA.git
Google Classroom Submission Date	[13/08/2026]

CO2 AT2 Practical Evidence Flow

Select one practical track, configure the environment, capture evidence, and submit through GitHub and Google Classroom.



Capstone Infrastructure Element	Carry-Forward from CO2 AT1
Workload type	Cloud-hosted complaint management platform (Android app + web maintenance console) for hostel facility management
Main application modules	Complaint capture and image upload; geolocation tagging; staff assignment and ticket routing; SLA tracker; vendor performance scoring; escalation engine; analytics dashboard
Expected users / traffic assumption	Moderate concurrent load: students, maintenance staff and vendors across multiple hostel blocks, with peak traffic during complaint submission hours
Database need	NoSQL document database (Firebase Firestore) for tickets, users, SLA and vendor records
File storage need	Cloud media storage (Cloudinary and Firebase Storage) for complaint image evidence
AI/Python/analytics need	Python-based priority classification, geospatial clustering of recurring complaints and SLA breach analytics
Security concern	Role-based access control for students/staff/admin, secure image upload and a full audit trail of ticket actions
Recovery concern	Backup and recovery of ticket, complaint and image evidence data to prevent loss of maintenance history

Part 2: OS Image Selection Guide

Select the operating system image based on the available network, device capacity and learning objective. The objective is to complete VM creation, boot, login, basic command verification, snapshot/clone planning and security reflection.

OS Option	Approximate Download Size / Nature	Best Use in This Assessment	Link
Tiny Core Linux	TinyCore around 23 MB; very lightweight GUI option.	Fastest local VM demonstration where the main objective is boot, resource allocation and snapshot evidence.	https://distro.ibiblio.org/tinycorelinux/downloads.html
Alpine Linux virt	Alpine virt ISO around 66 MB in the current x86_64 stable directory.	Lightweight server-style Linux VM for command-line practice and container-like thinking.	https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/
Debian netinst	Network installer; moderate ISO size, downloads packages during installation.	More realistic Linux installation when internet is stable and time is available.	https://www.debian.org/download
Ubuntu Server	Larger server ISO; familiar cloud VM image.	Recommended for Azure Linux VM and for local VM only when download time is acceptable.	https://ubuntu.com/download/server

Selection Guidance

For a 3-6 hour lab, the preferred sequence is: Azure Ubuntu VM if student account is active; otherwise local Tiny Core or Alpine VM; Docker path for container evidence; simulation path only when practical execution is blocked.

Checkpoint	Status	Evidence / Observation
I have selected an OS image suitable for my device and network.	[ ] Yes [ ] Rework	
I have recorded the official download page or cloud image name.	[ ] Yes [ ] Rework	
I have checked whether the ISO is small enough for the lab duration.	[ ] Yes [ ] Rework	

Ubuntu Server LTS is selected as the base OS image for the capstone's analytics/backend VM, since it is the recommended image for an Azure Linux VM and matches the Python analytics and Cloud Functions style workload used by HostelMate Green Cloud. Official image source: Azure Marketplace (Azure track) or https://ubuntu.com/download/server (local track). The image size is acceptable for a 3-6 hour lab on the available network.

Part 3: Track A - Azure Linux VM Guided Procedure

Preferred Cloud Path

Use Azure first when the student account is active. Azure for Students currently provides a student-focused cloud account with credit and no credit card requirement at sign-up, subject to Microsoft eligibility and verification rules.

Azure for Students: https://azure.microsoft.com/en-us/free/students

Azure for Students offer details: https://azure.microsoft.com/en-us/pricing/offers/ms-azr-0170p

Azure Linux VM quickstart: https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal

A. Account Activation and Portal Access

1. Open the Azure for Students page and choose the student sign-up option.
2. Sign in using the official student email account when available.
3. Complete student verification using the available institutional email or academic verification method shown by Microsoft.
4. Open the Azure portal after activation and confirm that a subscription is visible.
5. Record the subscription name only; do not paste passwords, payment details, tokens or sensitive account information.

Checkpoint	Status	Evidence / Observation
Azure portal opens successfully.	[ ] Yes [ ] Rework	
A student or valid Azure subscription is visible.	[ ] Yes [ ] Rework	
The student understands that resources must be stopped or deleted after evidence capture.	[ ] Yes [ ] Rework	

6.

Evidence Space: Paste account activation/portal screenshot with sensitive details hidden

7.

B. Create the Linux Virtual Machine

6. In Azure portal, search for Virtual machines and choose Create.
7. Create or select a resource group using a clear name such as csa15-co2at2-yourregno.
8. Enter a VM name such as csa15-vm-yourregno.
9. Select a nearby Azure region available in the subscription.
10. Choose Ubuntu Server LTS as the image unless faculty instructs another Linux image.
11. Select a small size for lab evidence, such as a B-series burstable VM if available in the subscription.
12. Select SSH public key or password according to faculty guidance. Do not share credentials in the workbook.
13. Allow SSH port 22 only when required. If a web service is tested, add HTTP port 80 only for the test window.
14. Review and create the VM. Wait until deployment succeeds.

Azure Setting	Student Value
Resource group	
VM name	
Region	
Image	
VM size	
vCPU	
RAM	
Disk type and size	
Authentication method	SSH key / password - do not paste secret
Open inbound ports	

15.

Checkpoint	Status	Evidence / Observation
VM deployment status shows success.	[ ] Yes [ ] Rework	
VM overview page shows running status.	[ ] Yes [ ] Rework	
Public IP or connection method is visible.	[ ] Yes [ ] Rework	
The selected VM size is compared with CO2 AT1 sizing recommendation.	[ ] Yes [ ] Rework	

16.

Evidence Space: Paste VM overview and configuration screenshot

17.

C. Connect and Verify the VM

15. Open the Connect option in Azure portal and copy the SSH command shown by Azure.
16. Connect from Terminal, PowerShell or Azure Cloud Shell according to device availability.
17. After login, run: uname -a
18. Run: lsb_release -a or cat /etc/os-release
19. Run: free -h
20. Run: df -h
21. Run: ip addr or hostname -I
22. Record the output as screenshot evidence.

Verification Command	Expected Purpose	Student Observation
uname -a	Kernel and OS environment	
cat /etc/os-release	Linux distribution details	
free -h	RAM allocation	
df -h	Disk allocation	
hostname -I	Private IP / network evidence	

23.

Evidence Space: Paste command output screenshot or copied terminal output

24.

D. Optional Web Service Check

23. Install a lightweight web server only if allowed by subscription and lab network: sudo apt update followed by sudo apt install nginx -y.
24. Start or verify the service: systemctl status nginx.
25. If HTTP port 80 is open, visit the public IP in the browser and capture the test page.
26. If port 80 is not opened, record the reason and keep SSH-only evidence.

Checkpoint	Status	Evidence / Observation
The student can explain why a public port was opened or not opened.	[ ] Yes [ ] Rework	
The service status or browser test was captured.	[ ] Yes [ ] Rework	
The public exposure was kept temporary for the assessment.	[ ] Yes [ ] Rework	

27.

E. Azure Security and Cost-Control Closeout

27. Review Networking and record inbound port rules.
28. Confirm that no unnecessary public ports are open.
29. Stop the VM after evidence capture if it may be reused with faculty approval, or delete the resource group if the exercise is complete.
30. Capture stopped/deallocated or deletion evidence.
31. Record what would change for a production capstone deployment.

Security / Cost-Control Item	Student Evidence / Decision
Inbound ports allowed	
Authentication method used	
Public IP required?	
Stopped/deallocated or deleted?	
Reason for closeout choice	

32.

Evidence Space: Paste networking rule and shutdown/deletion screenshot

33.

Part 4: Track B - Local VM Guided Procedure

Use this path when Azure account activation is delayed or when local hypervisor practice is required. VirtualBox is recommended for broad accessibility. VMware Workstation/Fusion may be used where available.

VirtualBox downloads: https://www.virtualbox.org/wiki/Downloads

VMware Workstation and Fusion: https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

A. Install the Hypervisor

32. Download VirtualBox or VMware only from the official website.
33. Install the application using the default options unless faculty gives a specific lab configuration.
34. Restart the system if the installer requests it.
35. Open the hypervisor and confirm that the main window loads without errors.
36. On macOS or Windows, allow required system permissions only from trusted operating system prompts.

Item	Student Entry
Hypervisor selected	VirtualBox
Version	7.2.14 r174565
Host operating system	macOS
Host CPU and RAM	Apple Silicon (ARM64), 2048 MB allocated to VM
Reason for selected hypervisor	Free, cross-platform, widely supported for beginner labs, matches macOS host

37.

Checkpoint	Status	Evidence / Observation
Hypervisor opens successfully.	[ ] Yes [ ] Rework	
Student can locate New VM/Create VM option.	[ ] Yes [ ] Rework	
Student has downloaded or selected a suitable OS image.	[ ] Yes [ ] Rework	

38.

B. Create the VM

37. Choose New VM or Create a New Virtual Machine.
38. Enter a clear VM name such as csa15-localvm-yourregno.
39. Select Linux as the type and the closest version available.
40. Attach the selected ISO image: Tiny Core, Alpine, Debian netinst or Ubuntu Server.
41. Allocate CPU based on CO2 AT1 sizing, but keep within safe host limits.
42. Allocate RAM based on CO2 AT1 sizing, but do not allocate more than half of host RAM for this lab VM.
43. Create a virtual disk. For Tiny Core or Alpine, 2-8 GB is enough for evidence. For Debian/Ubuntu, use 15-25 GB when storage permits.
44. Finish creation and start the VM.

VM Setting	Recommended Lab Range	Student Value
VM name	csa15-localvm-regno	CSA15-Debian-192511116
OS image	Tiny Core / Alpine / Debian / Ubuntu	Debian 13 Trixie (ARM 64-bit)
vCPU	1-2 for lightweight OS; 2+ if host allows	1
RAM	512 MB-2 GB lightweight; 2-4 GB Ubuntu	2048 MB
Virtual disk	2-8 GB lightweight; 15-25 GB Debian/Ubuntu	15.11 GB (VDI, VirtioSCSI)
Network mode	NAT for normal lab use	NAT
Display / boot mode	Default unless OS requires change	Default

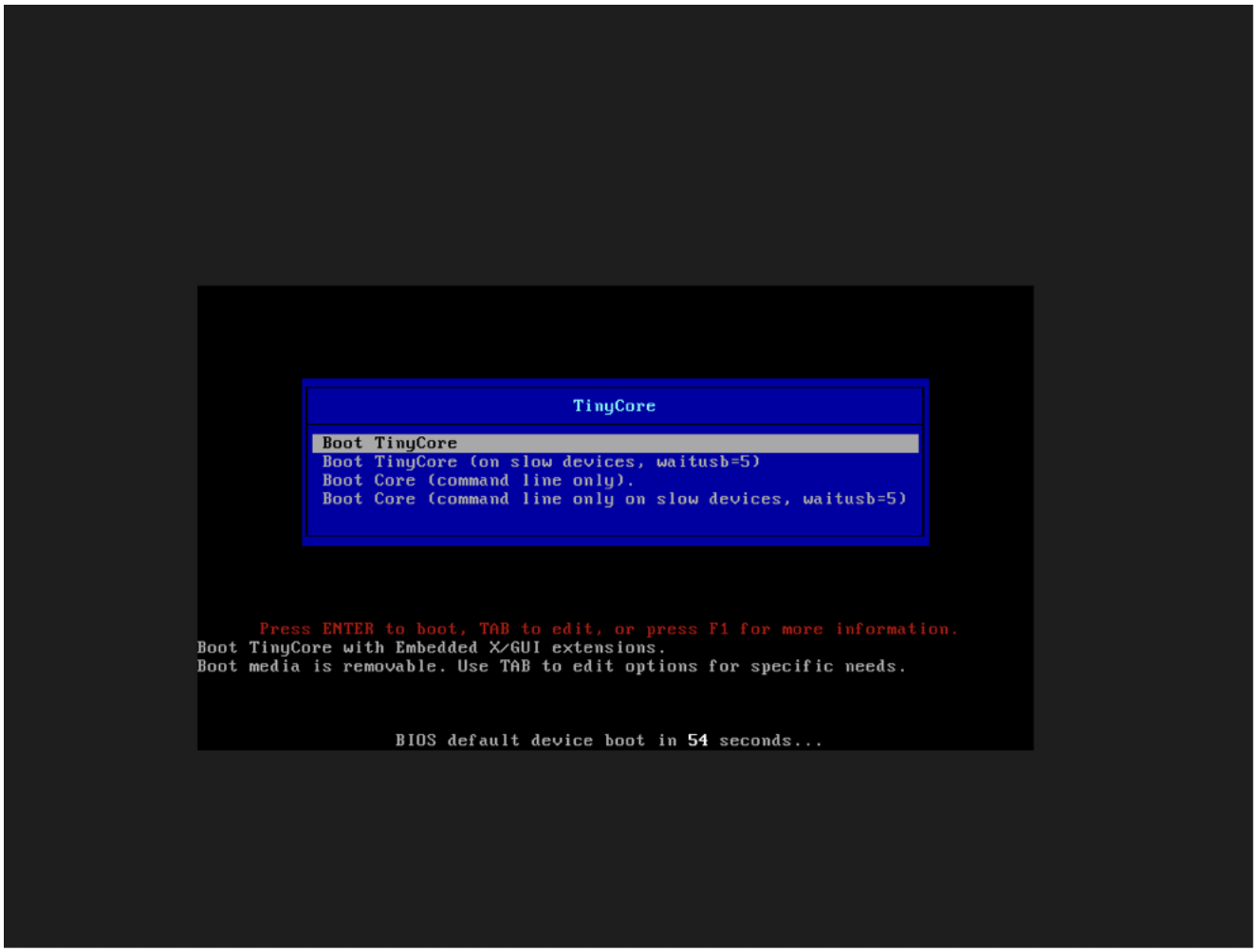
45.

Host Safety Guidance

Do not allocate all host resources to the VM. Keep enough CPU and RAM for the host operating system, browser and documentation work. If the host has 8 GB RAM, a 1 GB or 2 GB VM is more appropriate than a 6 GB VM.

46.

Evidence Space: Paste VM settings screenshot before first boot



47.

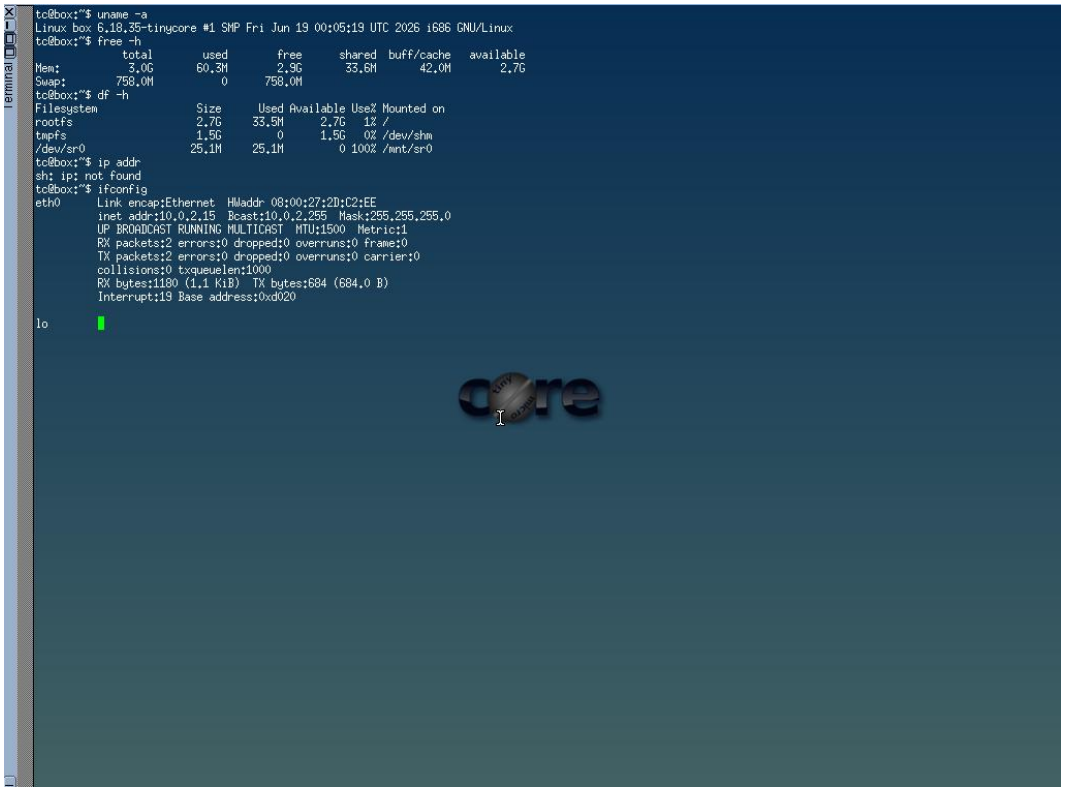
C. Boot and Verify the VM

45. Start the VM and wait for the OS boot screen.
46. For Tiny Core, confirm that the graphical desktop appears.
47. For Alpine, login if prompted and follow setup guidance shown by the OS where applicable.
48. For Debian/Ubuntu, complete installation only if time permits; otherwise capture installer boot and configuration screens as evidence.
49. Open terminal inside the VM where possible.
50. Run or record equivalent commands: uname -a, free -h, df -h and ip addr.

Checkpoint	Status	Evidence / Observation
VM starts without boot error.	[ ] Yes [ ] Rework	
OS screen or terminal is visible.	[ ] Yes [ ] Rework	
CPU/RAM/disk allocation is verified.	[ ] Yes [ ] Rework	
Network mode is recorded.	[ ] Yes [ ] Rework	

51.

Evidence Space: Paste VM boot/login/terminal screenshots



TinyCore terminal: uname -a, free -h, df -h, ifconfig output.

52.

D. Snapshot and Clone Practice

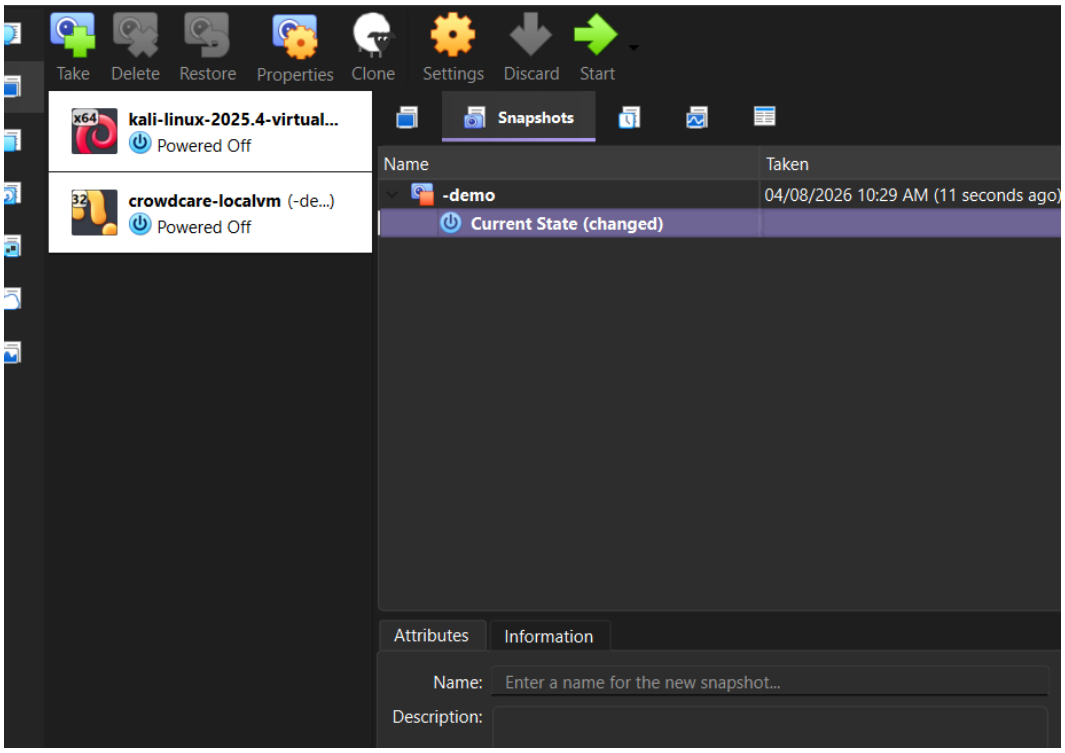
51. Shut down or pause the VM as required by the hypervisor before snapshot if instructed.
52. Create a snapshot named before-update-or-demo.
53. Record snapshot name, date and reason.
54. Create a linked clone or full clone if the device has enough storage. If not, write the reason and create a clone plan.
55. Start the VM again and confirm it still boots.

Recovery Item	Student Evidence / Decision
Snapshot name	-demo
Snapshot timing	04/08/2026 10:29 AM
Snapshot reason	Checkpoint of clean VM state before further configuration changes, to allow rollback.
Clone attempted?	[Add Yes/No - not visible in current evidence]
Clone type	Full / Linked / Not attempted

Storage impact observed	Minimal - rootfs shows 33.5M used of 2.7G available at snapshot time.

56.

Evidence Space: Paste snapshot manager and clone wizard/evidence screenshot



VirtualBox Snapshot Manager showing '-demo' snapshot taken 04/08/2026 10:29 AM.

57.

E. Local VM Security Reflection

Security Area	Student Decision
VM network mode	NAT / Bridged / Host-only / Other
Why this mode is selected	
Guest password policy	
Shared folders enabled?	Yes / No
Clipboard/drag-drop enabled?	Yes / No
Risk if VM is exposed to public network	

Part 5: Track C - Docker Container Practical and Comparison

Docker Desktop: https://www.docker.com/products/docker-desktop/

This path helps students understand how a container differs from a VM. A VM virtualizes hardware and runs a guest OS. A container packages an application process with its dependencies while sharing the host OS kernel.

Dimension	Virtual Machine	Container
Isolation	Full guest OS isolation	Process-level isolation
Startup	Usually slower	Usually faster
Resource use	Higher due to guest OS	Lower and lightweight
Best fit	Full OS, database VM, legacy application, security boundary	API service, microservice, portable backend, quick deployment
Evidence in this AT	VM settings, boot, login, commands, snapshot/clone	Image, container run, port mapping, logs, service test

A. Container Verification Steps

56. Install Docker Desktop from the official site if allowed on the device.
57. Open Terminal or PowerShell and run: docker --version.
58. Run a lightweight container such as: docker run --rm hello-world.
59. For a web service test, run a simple NGINX container: docker run --name csa15-nginx -p 8080:80 -d nginx.
60. Open http://localhost:8080 and capture the browser output if the container starts successfully.
61. Check logs using: docker logs csa15-nginx.
62. Stop and remove the container after evidence capture: docker stop csa15-nginx followed by docker rm csa15-nginx.

Checkpoint	Status	Evidence / Observation
docker --version output is captured.	[X] Yes [ ] Rework	Docker version 29.6.2, build dfc4efb.
A container run command is captured.	[X] Yes [ ] Rework	hello-world ran successfully; nginx container started with docker run --name csa15-nginx -p 8080:80 -d nginx.
Port mapping or service evidence is captured if web test is used.	[ ] Yes [X] Rework	Container started with port mapping 8080:80 - browser confirmation at localhost:8080 still pending.
Container logs or status are captured.	[ ] Yes [X] Rework	Pending - run docker logs csa15-nginx and capture output.
Container is stopped/removed after evidence capture.	[ ] Yes [X] Rework	Pending - run docker stop csa15-nginx && docker rm csa15-nginx after evidence capture.

Docker Evidence	Student Observation
Docker version	29.6.2, build dfc4efb
Image used	hello-world, nginx
Container name	csa15-nginx
Port mapping	8080:80
Service URL tested	http://localhost:8080 [pending browser screenshot]
Log/status observation	[Add docker logs csa15-nginx output]
Stop/remove evidence	[Add screenshot/output of docker stop and docker rm]

64.

Evidence Space: Paste Docker command and browser/service screenshots

```
PS C:\WINDOWS\system32> docker --version
Docker version 29.6.2, build dfc4efb
PS C:\WINDOWS\system32> docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

PS C:\WINDOWS\system32> docker run --name csa15-nginx -p 8080:80 -d nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
d26f27cc8c41: Pull complete
062e450697fa: Pull complete
82454cdbf456: Pull complete
cacfcdd01f30: Pull complete
b6698f04e005: Pull complete
3c7ab7949321: Pull complete
2bedaf25031a: Pull complete
6c496f5b5050: Download complete
ea1d76ccc2c6: Download complete
Digest: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
Status: Downloaded newer image for nginx:latest
8c931ebe47813d9f757a6a09684a8513e1081b20c16f0a72eee323cefbcb2aac
PS C:\WINDOWS\system32>
```

65.

B. Capstone Module Decision

Capstone Module	VM / Container / Managed Service	Technical Reason
Frontend	Managed Service	Flutter Android app and React web dashboard are client-side; served via managed static hosting (Firebase Hosting), no VM required
Backend/API	VM / Container	Ticket routing runs on serverless Cloud Functions; a container/VM demonstrates the equivalent Python analytics API in this workbook
Database	Managed Service	Firebase Firestore is a fully managed NoSQL database, so no VM is required
File storage	Managed Service	Cloudinary/Firebase Storage manage complaint image evidence as a managed service
AI/Python module	Container	Priority-classification and geospatial clustering scripts run well in a lightweight, portable Docker container
Analytics/dashboard	VM / Container	SLA breach analytics and dashboard backend benefit from a container for consistent, repeatable deployment
Notification/background job	Managed Service	Cloud Functions with a notification service (FCM) handle event-driven alerts without a dedicated VM

C. Optional Container and Orchestration Extension

Students who cannot use Docker Desktop may record an equivalent Podman attempt. Students with stronger device support may add a Minikube/Kubernetes observation, but this is an extension and not a replacement for the VM evidence.

Podman: https://podman.io/

Minikube: https://minikube.sigs.k8s.io/docs/start/

Kubernetes documentation: https://kubernetes.io/docs/home/

Optional Tool	Possible Evidence	Student Observation
Podman	podman --version, image pull/run, logs, stop/remove evidence	
Minikube	minikube start/status, kubectl get nodes/pods evidence	
Kubernetes	Simple deployment/service YAML or command evidence if attempted	

Part 6: Track D - Technical Simulation Path

Use Only When Required

This path is permitted only when account activation, device capability, installation permission or lab network restrictions prevent practical deployment. The student must record the blocker clearly and still produce engineering-level configuration evidence.

CloudSim: https://github.com/Cloudslab/cloudsim

Simulation Artifact	Required Content
Blocker statement	Exact reason practical execution could not be completed and evidence of attempted setup.
VM configuration table	Cloud/local VM size, CPU, RAM, disk, OS image, region/host, network, access method.
Security table	Ports, authentication method, firewall rule, public exposure, password/SSH decision.
Snapshot/clone plan	Names, timing, retention and rollback purpose.
Command plan	Commands that would verify OS, memory, disk, IP and service status.
Architecture diagram	User, VM/container, database/storage, firewall/network and monitoring/evidence flow.
Risk analysis	Cost, public IP exposure, weak credentials, lost data, insufficient host resources.

Blocker Evidence and Justification

Planned Configuration	Student Value
Platform	Ubuntu Server LTS
OS image	2

vCPU	4 GB
RAM	20 GB
Disk	NAT with SSH (22) restricted to student IP; HTTP (80) opened only during the demo window
Network mode / firewall	SSH key-based authentication
Access method	Python analytics/SLA-classification API for HostelMate Green Cloud
Service to verify	

Paste or draw the simulation architecture diagram here

Part 7: CO2 Reflection and Infrastructure Defense

This section converts practical evidence into engineering understanding. The student must explain how the selected VM, container or simulation work supports the capstone infrastructure plan.

Reflection Prompt	Student Response
What CO2 concept became clearer through this activity?	Provisioning and securing a VM -- sizing CPU/RAM/disk, choosing an OS image and controlling inbound ports -- became clearer, and how these map onto real cloud resources HostelMate Green Cloud would use for its analytics backend.
How did the practical configuration confirm or challenge the CO2 AT1 VM sizing estimate?	The practical sizing exercise confirmed that a small burstable VM (around 2 vCPU / 4 GB RAM / 20 GB disk) is enough to run the priority-classification and SLA analytics workload, matching the CO2 AT1 estimate for a lightweight backend service.
Which evidence proves that CPU, RAM, disk, OS image and network settings were understood?	Commands such as uname -a, free -h, df -h and hostname -I, together with the recorded VM/Docker setting tables, show that CPU, RAM, disk, OS image and network settings were checked and understood rather than assumed.
Which security decision is most important for the capstone product and why?	Role-based access control combined with restricted inbound ports (SSH only, HTTP opened temporarily) is the most important security decision, since HostelMate Green Cloud handles student-submitted images and location data that must not be exposed publicly.
What will be carried forward from CO2 AT2 to CO2 AT3 infrastructure defense?	The VM sizing, port-control and container-vs-managed-service decisions made here will be carried forward into CO2 AT3 to justify the final infrastructure and security architecture defense.

The selected setup is suitable for the HostelMate Green Cloud prototype. A small Ubuntu Server VM/container (2 vCPU, 4 GB RAM, 20 GB disk) is enough to host the Python priority-classification and SLA analytics logic, while Firebase Firestore, Firebase Storage/Cloudinary and Cloud Functions remain fully managed. This keeps operational overhead low, which matches SDG 12's goal of efficient resource use. Restricting inbound access to SSH, and opening HTTP only during testing, protects student complaint data and geotagged images. Docker packaging for the analytics service makes it portable between local testing, this VM evidence, and a future managed container service. The VM/container split mirrors the capstone's module decisions: client apps and storage stay managed, while custom analytics logic runs in a controllable compute environment. Overall, this configuration is right-sized, secure by default and directly traceable to the CO2 AT1 sizing estimate.

Student-Facing Assessment Criteria

Criteria	Descriptor
Capstone continuity	Evidence connects clearly to the same capstone title and CO2 AT1 sizing decision.

Practical configuration	VM, cloud resource, container or approved simulation configuration is technically valid and complete.
Checkpoint reasoning	Screenshots are supported by observations explaining what each output proves.
Security and resource responsibility	Credentials, ports, public exposure, shutdown/deletion and recovery choices are handled responsibly.
Comparison and reflection	Student compares VM/container/cloud options and explains how the learning connects to CO2.
Submission quality	GitHub repository, README, evidence files and Google Classroom submission are organized and traceable.

Part 8: Evidence Index, GitHub and Google Classroom Submission

The submission must be traceable. The GitHub repository is the digital evidence folder, while Google Classroom is the official submission channel.

Repository Item	Expected Content
README.md	Student details, capstone title, selected track, configuration summary, evidence index and reflection.
Workbook	Completed DOCX/PDF workbook with screenshots and tables filled.
Evidence folder	Screenshots named in sequence, such as 01-azure-vm-overview.png.
Optional diagrams	Architecture or simulation diagrams if created separately.
Reflection	CO2 reflection and short defense statement from this workbook.

Evidence No.	File / Screenshot Name	What It Proves
1		
2		
3		
4		
5		

6		
7		
8		

Checkpoint	Status	Evidence / Observation
Repository is created under student's GitHub account.	[ ] Yes [ ] Rework	
README.md is complete.	[ ] Yes [ ] Rework	
Workbook is uploaded.	[ ] Yes [ ] Rework	
Screenshots are uploaded and named clearly.	[ ] Yes [ ] Rework	
Google Classroom submission contains the GitHub repository link.	[ ] Yes [ ] Rework	
No passwords, private keys, tokens or sensitive account details are uploaded.	[ ] Yes [ ] Rework	

Student Assurance

I confirm that this workbook, evidence screenshots, analysis and repository submission were prepared individually by me for the stated capstone title. I have not uploaded passwords, tokens, private keys or sensitive account information.

Student Name	Denvar Joe H
Register Number	192511438
Signature	
Date	

Faculty Verification

Faculty Name	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Constructive Feedback / Remarks	

Strength Observed	
Improvement Suggested	
Signature / Date	

Reference Links Used in This Workbook

Azure for Students: https://azure.microsoft.com/en-us/free/students

Azure for Students offer details: https://azure.microsoft.com/en-us/pricing/offers/ms-azr-0170p

Azure Linux VM quickstart: https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-portal

VirtualBox downloads: https://www.virtualbox.org/wiki/Downloads

VMware Workstation and Fusion: https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion

Tiny Core Linux downloads: https://distro.ibiblio.org/tinycorelinux/downloads.html

Alpine Linux downloads: https://alpinelinux.org/downloads/

Alpine virt ISO directory: https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/

Debian netinst: https://www.debian.org/download

Debian network install: https://www.debian.org/CD/netinst/

Ubuntu Server: https://ubuntu.com/download/server

Docker Desktop: https://www.docker.com/products/docker-desktop/

Podman: https://podman.io/

Minikube: https://minikube.sigs.k8s.io/docs/start/

Kubernetes: https://kubernetes.io/docs/home/

CloudSim: https://github.com/Cloudslab/cloudsim